# Contents



# Week 1 — Introduction to Agentic AI & Research Methodology

**CO5151 · Advanced Agentic Artificial Intelligence** Learning outcomes touched: L.O.1.1, L.O.4.1 · Assessment touched: none directly (sets up SEM1/SEM2/PRJ) Duration: 150 minutes · Format: lecture + discussion + organizational segment

> **Schedule note:** the real HK261 timetable gives only 13 sessions, not 15. This content and the Week 2 notes (Reasoning & Planning) are both taught in one combined, flipped-classroom session — real Week 1, Wed 9 Sep 2026. Assign both documents as pre-reading before that class.

## Learning objectives

By the end of this session, students can:

1. Define an "agent" precisely enough to distinguish it from a chatbot, a workflow, and a script, and place a given system on an autonomy spectrum.
2. Decompose any agentic system into four functional components — perception/context, reasoning, memory, action/tools — wired together by a control loop, and map each seminar topic in the course onto this anatomy.
3. Apply a three-pass method to read a research paper, and reproduce the five-part structure (context–problem–method–results–limitations–open directions) expected of every seminar talk.
4. Ask an assumption-level critical question about a paper's claims, rather than a summary-level comprehension question.

---

## 1. From LLMs to autonomous agents

### 1.1 Why "agent" needs a definition

The word "agent" is used loosely in industry marketing (a chatbot with a system prompt is called an "agent"; so is a system that runs unsupervised for six hours across forty tool calls). For a research course this looseness is a liability: if the word means everything, critique becomes impossible, because there is no claim to hold a system to. We adopt a working definition for the semester:

> **An agent is a system in which a language model's output determines the next action taken in an environment, and that action's result is fed back into the model's context before the next decision.** The defining property is the *loop* — perceive, decide, act, observe, repeat — not any particular level of capability.

This definition deliberately excludes a single prompt-response call (no loop) and deliberately includes a two-step "look up a fact, then answer" tool call (a minimal but real loop). It is a floor, not a ceiling — most of the semester

is about what you add on top of the loop: memory across loops, multiple loops running concurrently, planning before acting, verification after acting.

### 1.2 A taxonomy of autonomy

It is useful to place any system you read about (or build) on a spectrum. Six points on that spectrum, from least to most autonomous:

| Level | Description | Example | Who is "in the loop" |
|---|---|---|---|
| 0 — Single call | One prompt, one completion, no loop | Autocomplete, a classification call | The developer, every call |
| 1 — Tool-augmented call | One reasoning step chooses zero or more tool calls, then answers | "What's the weather in Hanoi?" via one function call | The developer, every call |
| 2 — Workflow | A fixed, human-designed sequence or graph of LLM steps and tools; the *path* is hand-authored, only step content is generated | A prompt-chaining pipeline: extract → summarize → translate | The developer, at design time |
| 3 — Autonomous agent | The model decides *which* steps to take and *when to stop*, not just what to write inside a fixed step | ReAct-style agent solving a multi-hop question | The user, per task (approves/starts it) |
| 4 — Multi-agent system | Several autonomous agents (level 3) interact, each with its own loop, coordinated by protocol or a manager agent | AutoGen/MetaGPT-style software team | The user, per task; agents supervise each other |
| 5 — Open-ended / embodied | An agent operating continuously with no fixed task boundary, often in a real or persistent simulated environment | Voyager in Minecraft; a computer-use agent left running | Intermittent human oversight only |

Two things this table is for: (1) when you read a paper, decide which level its system actually operates at — many papers use the word "agent" for level-1 or level-2 systems, and the gap between the claimed and actual level is often the paper's weakest point; (2) when you scope your project, be explicit about which level you are building and evaluating at, since the evaluation bar is different at each level (level 2 is graded on correctness of a fixed pipeline; level 3+ is graded on decision quality under uncertainty).

**Anthropic's distinction (2024, "Building Effective Agents"), previewed here and covered in depth in S1-7**: *workflows* (level 2) are predictable and consistent for well-defined tasks; *agents* (level 3+) are appropriate when flexibility and model-driven decision-making are needed at scale, at the cost of latency and unpredictability. Their guidance — "find the simplest solution possible, and only increase complexity when needed" — is a recurring theme this semester and should be in the back of your mind for the project's architecture decisions.

### 1.3 A short research timeline (2022–2026)

Orientation, not exam material — a map of where this semester's topics sit in time:

| Year | Milestone |
|---|---|
| 2022 | Chain-of-Thought prompting (Wei) — reasoning emerges from prompting alone |
| 2022 | InstructGPT / RLHF (Ouyang) — models start reliably following instructions |
| 2023 | ReAct (Yao) — interleave reasoning traces with actions |
| 2023 | Toolformer (Schick), Reflexion (Shinn), Generative Agents (Park), AutoGen/MetaGPT/CAMEL |
| 2023 | Tree of Thoughts (Yao) — search over reasoning paths |
| 2024 | Model Context Protocol (Anthropic) — standardized tool/context interface |
| 2024 | SWE-bench / SWE-agent — coding agents as a measurable research object |
| 2024 | AgentBench / GAIA / WebArena — agent evaluation matures as its own subfield |
| 2024 | "Building Effective Agents" (Anthropic) — workflows-vs-agents design guidance crystallizes |
| 2024 | AgentDojo / indirect prompt injection formalized as a benchmarked attack surface |
| 2024 | GRPO / DeepSeekMath — reinforcement learning with verifiable rewards at scale |
| 2025 | DeepSeek-R1, o1/o3-class reasoning models — RL-trained reasoning replaces prompted CoT |
| 2025 | Computer-use agents (Anthropic, OpenAI Operator) — agents act on real GUIs |
| 2025 | Agent2Agent (A2A) protocol — standardizing agent-to-agent (not just agent-to-tool) communication |
| 2025 | CaMeL and "design patterns for securing agents" — injection defenses move from detection to architecture (capability-based control, information-flow tracking) |

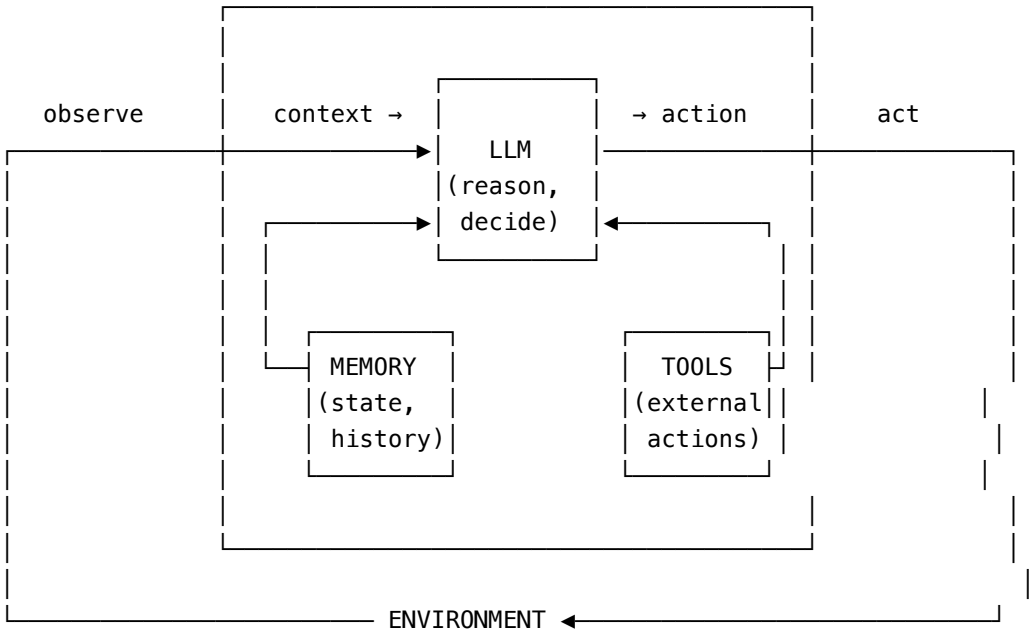
Notice the shape of this timeline: 2022–2023 is dominated by *prompting tricks* that elicit behavior from a frozen model (CoT, ReAct, Reflexion all work on off-the-shelf models). 2024 is dominated by *infrastructure* — protocols and benchmarks that make agents buildable and measurable at scale. 2025 is dominated by *training* — RL that bakes the behavior into the model rather than eliciting it via prompting, and *security architecture* that stops trying to filter bad text and starts constraining what an agent is capable of doing regardless of what it is told. This shift — prompting → tooling → training/architecture — is one lens for reading every paper you'll encounter this semester: ask which phase a given technique belongs to, and whether a later phase has since subsumed it.

---

## 2. Anatomy of an agent

### 2.1 The four components

Every system covered this semester — regardless of how it's marketed — can be described with four components and one loop connecting them:




- **Perception / context** — what the model sees at decision time: the current task, the conversation so far, tool results, retrieved documents. This is not passive; *what you choose to put in context, and in what order,* is itself a design decision (see §3 below and S1-3/S1-4 in Seminar 1).
- **Reasoning** — the LLM call(s) that turn context into a decision: what to do next, or a final answer. Everything in S1-1 (CoT, ReAct, Reflexion, ToT) and S1-6 (self-improvement) is a technique for structuring *this* component.
- **Memory** — state that persists *across* loop iterations and, for long-lived agents, across sessions. Short-term memory is usually just "more context"; long-term memory requires a separate store with its own write and retrieval policy (S1-3).
- **Action / tools** — how the agent affects (or queries) the world outside its own context window: function calls, code execution, web requests, database queries, MCP-exposed tools (S1-2, and Week 3 of this course).

This is a compressed version of Sumers et al.'s **CoALA** (Cognitive Architectures for Language Agents, TMLR 2024) framework, which additionally separates *internal* actions (writing to memory, reasoning) from *external* actions (calling a tool, sending a message) and introduces a **decision procedure** that selects among candidate

actions each cycle. You are not required to use CoALA's exact vocabulary, but you are expected to be able to place any technique from this course's reading list into one of these boxes — that placement is often the first useful thing to say in a critique ("this paper's contribution is entirely in the reasoning box; it says nothing about memory or tools").

### 2.2 The control loop, made concrete

In pseudocode, almost every agent in the literature is a variant of:

```
def agent_loop(task, tools, memory, max_steps):
    context = build_initial_context(task, memory)
    for step in range(max_steps):
        decision = llm(context)                      # REASONING
        if decision.is_final_answer:
            return decision.answer
        result = execute(decision.action, tools)     # ACTION → ENVIRONMENT
        memory.maybe_write(step, decision, result)   # MEMORY
        context = update_context(context, decision, result)  # PERCEPTION for next step
    return fail("max steps exceeded")
```

What differs between ReAct, Reflexion, ToT, and a multi-agent system is *what happens inside this loop* — how `decision` is produced (single forward pass vs. search over branches), whether `memory` is trivial (nothing) or structured (a vector store with a write policy), and whether there is one loop or several running concurrently and exchanging messages. Keeping this skeleton in mind while reading is the fastest way to see what is genuinely new in a paper versus what is a relabeling of an existing idea.

### 2.3 Where this semester's topics fit

| Component / loop property | Seminar 1 topics | Seminar 2 topics |
|---|---|---|
| Reasoning (single loop) | S1-1 Reasoning & Planning, S1-6 Self-improvement & Reflection | S2-6 RL for Agents (trains this component) |
| Perception / context | S1-4 Retrieval-Augmented Agents | S2-7 Computer-use/GUI/Embodied (perception is pixels/DOM, not just text) |
| Memory | S1-3 Agent Memory | S2-4 Advanced Security (memory *poisoning*) |
| Action / tools | S1-2 Tool Use & MCP, S1-5 Coding Agents | S2-3/S2-4 Security (tools as attack surface) |
| Multiple loops / orchestration | S1-7 Design Patterns & Orchestration | S2-1/S2-2 Multi-Agent Systems & Communication |
| Evaluating the whole loop | S1-8 Evaluation & Benchmarking | S2-5 Safety & Alignment, S2-8 Production & Governance |

Use this table when you're deciding how to frame your own seminar talk's opening ("this paper is about the reasoning component specifically, and assumes the tool and memory components are solved / out of scope") — naming the scope precisely is itself a form of critical analysis, not just organization.

---

## 3. How to read, present, and critique a research paper

### 3.1 The three-pass method

Reading a systems/ML paper start-to-finish, linearly, is the slowest possible way to extract what you need, and it's why "I read it but don't understand it" happens. Use three passes instead (adapted from Keshav's classic method for this course's needs):

**Pass 1 (5–10 min) — the shape of the paper.** Read title, abstract, headings, figures, and the conclusion. Do not read the method or results in detail yet. After this pass you should be able to answer: *what problem is this solving, and what is the one-sentence claim of the contribution?* If you can't answer that after pass 1, the paper's writing is unclear, or you've misjudged the venue/scope — either way, note it; that's already a critical observation.

**Pass 2 (30–60 min) — the argument.** Read the paper in order, but skip proofs/derivations and treat figures as the primary evidence. Note: what is the *baseline* they compare against? What is the *metric*? Is the metric measuring what the abstract claims, or something narrower? What experimental knobs (model size, number of seeds, prompt variants) are and are not varied? After this pass, sketch the method as your own diagram or pseudocode from memory — if you can't, you haven't understood the mechanism, only the marketing.

**Pass 3 (only for your core papers, 1–2+ hours) — reconstruction.** Attempt to re-derive the method's key steps, or actually run the released code/replicate a small version. This is where you find the gap between what the paper says and what it does — e.g., a "zero-shot" claim that turns out to rely on few-shot exemplars in an appendix, or a benchmark number that only holds for one model family.

For this course: pass 1 on every paper in the reading list (so you can follow every seminar's Q&A), pass 2 on your own topic's core + extension papers, pass 3 on at least the single most important paper for your seminar talk and for your project's chosen method.

### 3.2 Presentation structure (what every seminar talk must do)

Every 30-minute talk this semester — Seminar 1 and Seminar 2 alike — follows the same five-part structure, reiterated from the syllabus:

1. **Context & problem** — why does this topic matter, what question do the papers answer, what would the world look like if this problem were unsolved (it usually still partly is)?
2. **Survey of main approaches** — explain the *mechanism*, not just the name. "ReAct interleaves reasoning and acting" is a summary; showing an actual trace and explaining *why* interleaving beats reason-then-act is a survey.
3. **Critical analysis** — this is 25% of your seminar grade and the single most differentiating part of a talk. See §3.3.
4. **Demo / example** — a worked trace, a small live demo, or a concrete input/output example. Abstract claims become checkable when you show one real run.
5. **Relation to projects & open directions** — connect explicitly to your own project's architecture, and name what remains unsolved (not "more research is needed" — a specific, falsifiable open question).

### 3.3 What counts as critical analysis (vs. summary)

The single biggest scoring failure mode in past cohorts of similar courses is presenting an accurate, well-organized *summary* and calling it analysis. A summary answers "what does the paper say?" Analysis answers one of these instead:

- **Assumptions.** What does the method require to be true about the environment, the task, or the model, that isn't stated as a limitation? (E.g., ReAct assumes actions are cheap and observations are immediately available — expensive or slow actions change the cost-benefit entirely.)

- **Baseline validity.** Is the comparison fair? Same number of forward passes / tokens / dollars spent across methods? A method that "beats" a baseline by using 10× more inference compute is not obviously better — it's a different point on a cost-quality curve.
- **Metric validity.** Does the reported number measure the paper's actual claim? (E.g., "success rate" on a benchmark where the LLM judge itself has a known bias, or pass@1 reported where pass@k with retries would tell a different story.)
- **Generalization.** Would this result hold on a different model family, a different domain, a slightly harder version of the task? Papers rarely test this; you can point out that they don't.
- **Reproducibility & contamination.** Is the benchmark's test data plausibly in the model's training set? Are seeds/variance reported, or is this a single run?
- **Comparison across papers.** Two papers in the same topic sheet often make *implicitly conflicting* claims (e.g., one topic's papers claim self-correction works, another shows it doesn't under matched conditions) — surfacing and explaining the conflict is exactly the kind of synthesis this course wants.

A useful habit: for every core paper, write one sentence that starts with "This paper's result would not hold if…" — that sentence is usually most of a critical-analysis slide.

### 3.4 Worked example: a live critique of ReAct (Yao et al., ICLR 2023)

Use this as the model the instructor performs live in Week 1, and as a template for your own talks.

- **Claim**: interleaving reasoning traces ("thoughts") with actions and observations improves task success over reasoning-only (CoT) or acting-only baselines on knowledge-intensive QA and interactive decision-making tasks.
- **Mechanism, reconstructed**: at each step the model generates a `Thought` (free text), then an `Action` (a tool call from a fixed set, e.g., `Search[query]`, `Lookup[string]`, `Finish[answer]`), receives an `Observation` (the tool's result appended to context), and repeats. The thought is not executed — it exists purely to condition the next action on an explicit intermediate reasoning step.
- **Assumption check**: the action space is small, fixed, and each action is cheap and fast (a search API call). What happens if actions are expensive (e.g., each action is a paid transaction) or slow (e.g., each action takes minutes)? The paper doesn't test this — a strong critique names it as an open assumption rather than a settled result.
- **Baseline check**: ReAct is compared against CoT-only and Act-only prompting at (roughly) matched numbers of few-shot exemplars — a reasonably fair comparison — but not against a much simpler "retrieve-then-answer" pipeline at matched *token budget*, which later work (retrieval-augmented baselines) suggests can be competitive on some of the same benchmarks. Naming what comparison is *missing* is itself valid critique.
- **Generalization check**: results are reported on specific benchmarks (HotpotQA, FEVER, ALFWorld, WebShop) with specific action spaces designed for those benchmarks. Whether "interleave thought and action" remains beneficial with today's much stronger base models (which already do implicit step-by-step reasoning without being asked) is an open, testable question you could raise.
- **Verdict for the semester**: ReAct is the ancestor of nearly every agent framework covered in S1-2 and S1-7 (the `Thought → Action → Observation` loop is literally what LangGraph, most agent SDKs, and your own project's control loop will implement) — but treat its specific empirical numbers as dated and its action-cost assumption as the thing to test, not accept, in your own systems.

---

## 4. Discussion questions for this session

Use these to prime in-class discussion and to model the kind of question you should be asking (and be asked) all semester:

1. Pick a system you've used that is marketed as "an AI agent." Where does it sit on the §1.2 spectrum, and what evidence would you need to move it up or down a level?
2. For the anatomy in §2.1: name one technique from the reading list that touches *three or more* components simultaneously (a genuinely integrative system) versus one that touches only one.
3. The timeline in §1.3 argues that the field moved from prompting → tooling → training/architecture. What would evidence *against* this narrative look like — i.e., what would you expect to see if the phases were not actually sequential but simply concurrent research communities?
4. Take one sentence from any paper's abstract in your assigned topic and rewrite it as a falsifiable claim with a stated scope ("this holds for X under condition Y") — then identify what experiment in the paper does or doesn't support that scoped claim.

## Further reading

- Sumers, Yao, Narasimhan, Griffiths — *Cognitive Architectures for Language Agents* (TMLR 2024) — the CoALA framework referenced in §2.1.
- Wang et al. — *A Survey on Large Language Model based Autonomous Agents* (Frontiers of Computer Science 2024) — broader survey covering the whole landscape in §1.3.
- Xi et al. — *The Rise and Potential of Large Language Model Based Agents: A Survey* (arXiv 2023).
- Kapoor et al. — *AI Agents That Matter* (arXiv 2024 / TMLR 2025) — a methodology-focused critique of agent evaluation practice; previews the concerns in §3.3 and in S1-8.
- Yao et al. — *ReAct: Synergizing Reasoning and Acting in Language Models* (ICLR 2023) — the paper critiqued in §3.4; read it before class if you weren't assigned S1-1.